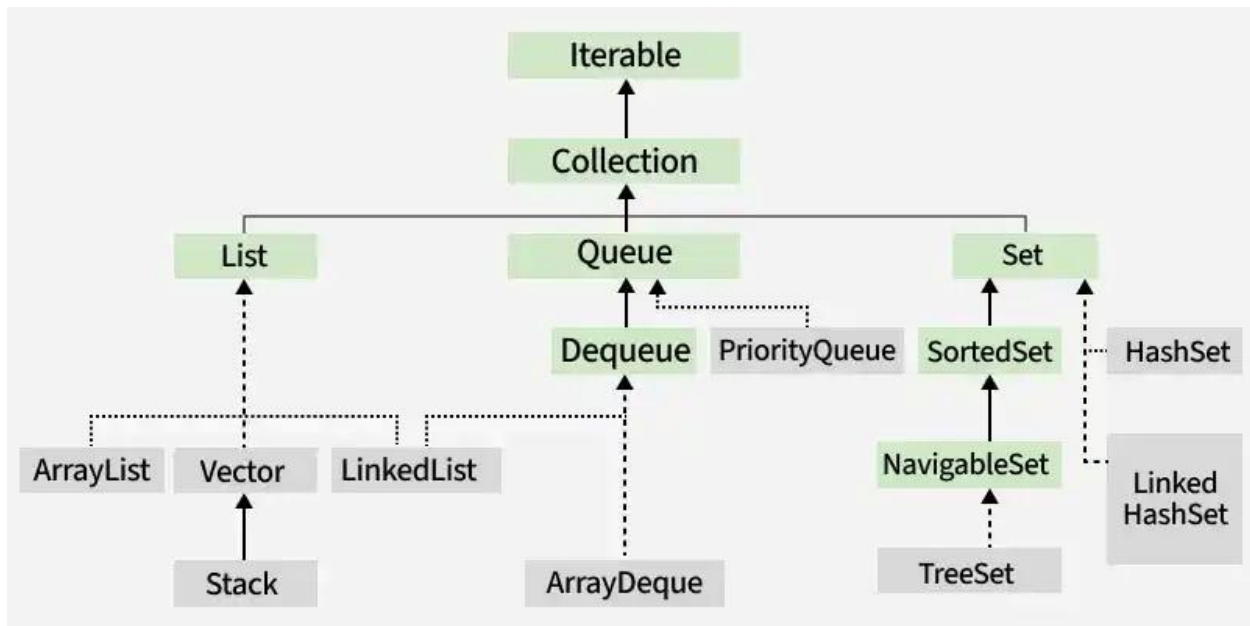


Java Collections Tutorial

Last Updated : 1 Apr, 2026

Java Collection Framework (JCF) is a set of classes and interfaces that provide ready-made data structures to store and manipulate groups of objects efficiently.

- Java provides collection interfaces like List, Set, Map, and Queue, with ready-made classes such as ArrayList, HashSet, HashMap, and PriorityQueue, so you don't have to write data-handling code from scratch.
- The Collection Framework improves productivity by making code more reusable, maintainable and faster to develop.



Features of Java Collection Framework

- Provides ready-to-use data structures (e.g., ArrayList, HashSet, HashMap).
- Offers interfaces (Collection, List, Set, Map, Queue) to define standard behaviors.
- Supports dynamic resizing, unlike arrays with a fixed size.
- Includes algorithms (sorting, searching, iteration) via the Collections utility class.
- Improves code reusability and performance by reducing boilerplate code.

Real World Example

Suppose you are managing a library where thousands of books need to be stored, searched and retrieved. Instead of arranging them manually in different shelves every time, you use a catalog system. Similarly, the Java Collection Framework acts as a catalog that organizes and manages objects efficiently.

Example

Java Program to illustrate the use of ArrayList (dynamic array) in the Collection Framework

```
import java.util.*;
public class Geeks {
    public static void main(String[] args) {

        // Creating a List of Strings using ArrayList (dynamic array)
        List<String> list = new ArrayList<>();

        // Adding elements to the ArrayList
        list.add("Java");
        list.add("Python");
        list.add("C++");
    }
}
```

```
// Printing the elements of the ArrayList
System.out.println("Programming Languages:");

// Enhanced for-loop to iterate through the list
for (String lang : list) {
    System.out.println(lang);
}
}
```

Output

Programming Languages:

Java

Python

C++

1. Core Interfaces

The foundation of the Collections Framework is built on interfaces like Collection, List, Set, Queue, Deque and Map. They define the behavior of different collection types and serve as a blueprint for implementations.

- [Collection Interface](#)
- [List Interface](#)
- [Set Interface](#)
- [Queue Interface](#)
- [Deque Interface](#)
- [Map Interface](#)

Collection Interface in Java

Last Updated : 1 Apr, 2026

- The Collection interface is the root of the Java Collections Framework, defined in the [java.util package](#). It represents a group of individual objects as a single unit and provides basic operations for working with them.
 - **Dynamic in Nature:** Collections can automatically grow or shrink in size, unlike arrays that have a fixed length.
 - **Stores Homogeneous Objects:** With generics, collections store elements of a specified type, ensuring type safety. Without generics (raw collections), they can hold heterogeneous objects, but this is discouraged.

- **Easy to Use:** Provides convenient methods such as add(), remove(), and clear() to manage elements effortlessly.
- **Efficient Traversal:** Allows easy access and processing of elements using loops or iterators.



```
import java.util.*;

public class GFG {

    public static void main(String[] args) {

        // Creating a Collection of String type using ArrayList
        // implementation
        Collection<String> fruits = new ArrayList<>();

        // Adding elements to the collection
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");

        // Removing an element from the collection
        fruits.remove("Banana");

        // Displaying the collection after removal
        System.out.println("After Removal: " + fruits);
    }
}
```

Output

```
After Removal: [Apple, Mango]
```

Collection Interface Declaration

public interface Collection<E> extends Iterable<E>

Here, E represents the type of elements stored in the collection.

Object Creation of Collection Interface

```
Collection<String> fruits = new ArrayList<>();
```

In Java, we cannot create an object of an interface directly. Instead, we create an object of the ArrayList class that implements the interface and assign it to the interface reference.

Sub-Interfaces of Collection Interface

The subinterfaces of Collection, also called collection types, are:

1. List

- [List](#) represents an ordered collection that allows duplicates.
- Elements can be accessed by index.
- **Implementing Classes:** [ArrayList](#), [LinkedList](#), [Vector](#), [Stack](#).
- **Declaration:**

```
public interface List<E> extends Collection<E>
```

2. Set

- [Set](#) represents an unordered collection with no duplicate elements.
- **Implementing Classes:** [HashSet](#), [TreeSet](#), [LinkedHashSet](#), EnumSet, CopyOnWriteArraySet.
- **Declaration:**

```
public interface Set<E> extends Collection<E>
```

5. Queue

- [Queue](#) represents a collection following FIFO (First-In-First-Out) order.
- **Implementing Classes:** [PriorityQueue](#), [Deque](#), [ArrayDeque](#), [LinkedList](#)
- **Declaration:**

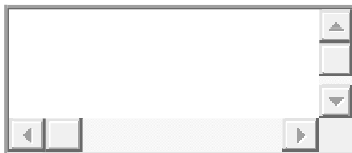
```
public interface Queue<E> extends Collection<E>
```

Operations on Collection Objects

The Collection interface provides several operations to manipulate data efficiently. Let's see the most common operations using the ArrayList implementation class.

1. Adding Elements

We can add elements using the add(E e) method for a single element or addAll(Collection c) to add multiple elements.



```
import java.util.*;

public class Geeks{

    public static void main(String[] args){

        // Creating a collection using ArrayList implementation
        Collection<Integer> numbers = new ArrayList<>();

        // Adding individual elements
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        // Adding another collection
        Collection<Integer> moreNumbers = new ArrayList<>();
        moreNumbers.add(40);
        moreNumbers.add(50);

        numbers.addAll(moreNumbers);

        System.out.println("After adding elements: " + numbers);
    }
}
```

Output

```
After adding elements: [10, 20, 30, 40, 50]
```

2. Removing Elements

Elements can be removed using `remove(E e)` or `removeAll(Collection c)` methods.



```
import java.util.*;

public class Geeks{

    public static void main(String[] args){

        Collection<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Mango");
        fruits.add("Orange");

        System.out.println("Initial Collection: " + fruits);

        // Remove a specific element
        fruits.remove("Mango");
        System.out.println("After removing Mango: " + fruits);

        // Remove all elements present in another collection
        Collection<String> toRemove = new ArrayList<>();
        toRemove.add("Apple");
        toRemove.add("Banana");

        fruits.removeAll(toRemove);
        System.out.println("After removeAll(): " + fruits);
    }
}
```

Output

```
Initial Collection: [Apple, Banana, Mango, Orange]
After removing Mango: [Apple, Banana, Orange]
After removeAll(): [Orange]
```

3. Accessing Elements

Although the Collection interface doesn't provide index-based access, its sub-interface List (implemented by ArrayList) allows retrieving elements using the `get(int index)` method.



```
import java.util.*;

public class Geeks{

    public static void main(String[] args){

        // Using List reference for index-based access
        List<String> colors = new ArrayList<>();
        colors.add("Red");
        colors.add("Green");
        colors.add("Blue");

        System.out.println("Colors List: " + colors);

        // Accessing elements by index
        String firstColor = colors.get(0);
        String lastColor = colors.get(colors.size() - 1);

        System.out.println("First Color: " + firstColor);
        System.out.println("Last Color: " + lastColor);
    }
}
```

Output

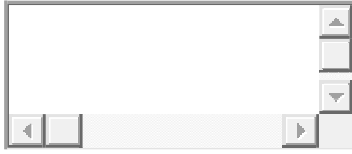
```
Colors List: [Red, Green, Blue]
First Color: Red
Last Color: Blue
```

4. Iterating over a Collection

To iterate over a collection we used Different types of Cursor

1. Iterator

- The Iterator interface allows traversal in one direction (forward).
- You can safely remove elements during iteration using remove().



```
import java.util.*;

public class IteratorExample {
    public static void main(String[] args) {
        Collection<String> fruits = new
        ArrayList<>(Arrays.asList("Apple", "Banana", "Mango"));

        Iterator<String> it = fruits.iterator();
        while (it.hasNext()) {
            String fruit = it.next();
            if (fruit.equals("Banana")) {
                it.remove(); // safely removes "Banana" from
collection
            }
        }
        System.out.println(fruits); // Output: [Apple, Mango]
    }
}
```

Output

```
[Apple, Mango]
```

2. ListIterator

- ListIterator extends Iterator and is available for List implementations (ArrayList, LinkedList, etc.).
- Supports bidirectional traversal (forward and backward).
- Allows adding, removing, and replacing elements during iteration.



```
import java.util.*;

public class ListIteratorExample {
    public static void main(String[] args) {
```

```

        List<String> colors = new
ArrayList<>(Arrays.asList("Red", "Green", "Blue"));

        ListIterator<String> listIt = colors.listIterator();

        // Forward traversal
        System.out.print("Forward: ");
        while (listIt.hasNext()) {
            System.out.print(listIt.next() + " ");
        }

        System.out.print("\nBackward: ");
        while (listIt.hasPrevious()) {
            System.out.print(listIt.previous() + " ");
        }
    }
}

```

Output

```

Forward: Red Green Blue
Backward: Blue Green Red

```

Removing During Iteration

- Using `Iterator.remove()` or `ListIterator.remove()` is the correct way to remove elements while iterating.
- Directly using `collection.remove(element)` inside a loop will throw `ConcurrentModificationException`.

Methods of Collection Interface

Method	Description
<u>add(E e)</u>	Adds an element to the collection (optional).
<u>addAll(Collection<? extends E> c)</u>	Adds all elements from another collection.
<u>clear()</u>	Removes all elements.

Method	Description
<u><code>contains(Object o)</code></u>	Checks if the collection contains the specified element. Takes Object as a parameter because it uses the <code>equals(Object obj)</code> method to check equality.
<u><code>containsAll(Collection<?> c)</code></u>	Checks if all elements exist.
<u><code>remove(Object o)</code></u>	Removes the first occurrence of the specified element. Takes Object as a parameter because it uses <code>equals(Object obj)</code> to identify the element to remove.
<u><code>removeAll(Collection<?> c)</code></u>	Removes all elements present in another collection.
<u><code>retainAll(Collection<?> c)</code></u>	Retains only elements present in another collection.
<u><code>removeIf(Predicate<? super E> filter)</code></u>	Removes elements satisfying a predicate.
<u><code>size()</code></u>	Returns the number of elements.
<u><code>isEmpty()</code></u>	Checks if the collection is empty.
<u><code>iterator()</code></u>	Returns an iterator over the elements.
<u><code>stream()</code></u>	Returns a sequential stream.
<u><code>parallelStream()</code></u>	Returns a parallel stream.


```

{
    // Creating a List of Strings using ArrayList
    List<String> li = new ArrayList<>();

    // Adding elements in List
    li.add("Java");
    li.add("Python");
    li.add("DSA");
    li.add("C++");

    System.out.println("Elements of List are:");

    // Iterating through the list
    for (String s : li) {
        System.out.println(s);
    }
}
}

```

Output

```

Elements of List are:
Java
Python
DSA
C++

```

Explanation: This Java program demonstrates how to create a List using ArrayList, add elements to it, and iterate through the list to print each element. It uses an enhanced for loop to display all the stored programming languages.

Declaration of Java List Interface

```
public interface List<E> extends Collection<E> { }
```

To use a List, we must instantiate a class that implements it. Example classes that implement it are ArrayList and LinkedList

```
List<Type> list = new ArrayList<Type>();
```

Hierarchy of List Interface

It extends the Collection interface.

The common implementation classes of the List interface are:

- **ArrayList**: It is implemented using resizable array, offers fast random access but slower insert/delete.
- **LinkedList**: It is implemented using Doubly-linked list, efficient for frequent insertions and deletions.
- **Vector**: It is implemented using Legacy synchronized dynamic array, thread-safe but slower.
- **Stack**: It is implemented using a LIFO (Last-In-First-Out) subclass of Vector for stack operations.

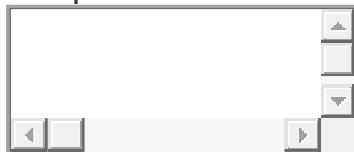
Java List - Operations

List can be used only with a class that implements this interface. Now, let's see how to perform a few frequently used operations on the List.

1. Adding Elements

To add an element to the list, we can use the `add()` method. This method is overloaded to perform multiple operations based on different parameters.

- **`add(Object o)`**: This method is used to add an element at the end of the List.
- **`add(int index, Object o)`**: This method is used to add an element at a specific index in the List



```
import java.util.*;

class Geeks {

    public static void main(String args[])
    {
        List<String> al = new ArrayList<>();

        // Adding elements to object of List interface Custom
elements
        al.add("Geeks");
        al.add("Geeks");
        al.add(1, "For");

        System.out.println(al);
    }
}
```

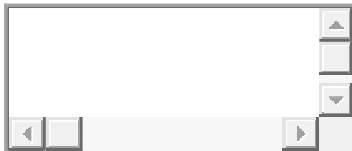
Output

```
[Geeks, For, Geeks]
```

Note: If we try to add element at index 1 before adding elements at index 0 it will throw an error. It is always recommended to add elements in a particular index only when the size is defined or to add them sequentially.

2. Updating Elements

To update an element in a list, use the `set()` method with the target index and the new value. Since List is indexed, the element is replaced at the specified position.



```
import java.util.*;

class Geeks {

    public static void main(String args[])
    {
        // Creating an object of List interface
        List<String> al = new ArrayList<>();

        // Adding elements to object of List class
        al.add("Geeks");
        al.add("Geeks");
        al.add(1, "Geeks");

        System.out.println("Initial ArrayList " + al);

        // Setting (updating) element at 1st index using set()
method    al.set(1, "For");

        System.out.println("Updated ArrayList " + al);
    }
}
```

Output

```
Initial ArrayList [Geeks, Geeks, Geeks]
Updated ArrayList [Geeks, For, Geeks]
```

3. Searching Elements

Searching in a List can be done using [indexOf\(\)](#), [lastIndexOf\(\)](#) methods.

- **indexOf(Object o):** It returns the index of the first occurrence of the specified element in the list or -1 if the element is not found
- **lastIndexOf(Object o):** It returns the index of the last occurrence of the specified element in the list or -1 if the element is not found



```
import java.util.*;

class Geeks {

    public static void main(String[] args)
    {
        // create a list of integers
        List<Integer> al = new ArrayList<>();

        // add some integers to the list
        al.add(1);
        al.add(2);
        al.add(3);
        al.add(2);

        // use indexOf() to find the first occurrence of an
        element in the list
        int i = al.indexOf(2);

        System.out.println("First Occurrence of 2 is at Index:
"+i);

        // use lastIndexOf() to find the last occurrence of an
        element in the list
        int l = al.lastIndexOf(2);

        System.out.println("Last Occurrence of 2 is at Index:
"+l);
    }
}
```


Output

```
First Occurrence of 2 is at Index: 1
Last Occurrence of 2 is at Index: 3
```

4. Removing Elements

To remove an element from a list, we can use the [remove\(\)](#) method. This method is overloaded to perform multiple operations based on different parameters.

- **remove(Object o):** Removes the first occurrence of the specified object from the list.
- **remove(int index):** Removes the element at the specified index and shifts subsequent elements left.



```
import java.util.ArrayList;
import java.util.List;

class Geeks {

    public static void main(String args[])
    {

        // Creating List class object
        List<String> al = new ArrayList<>();

        // Adding elements to the object Custom inputs
        al.add("Geeks");
        al.add("Geeks");

        // Adding For at 1st indexes
        al.add(1, "For");

        System.out.println("Initial ArrayList " + al);

        // Now remove element from the above list present at 1st
index
        al.remove(1);

        System.out.println("After the Index Removal " + al);
```

```

        // Now remove the current object from the updated List
        al.remove("Geeks");

        System.out.println("After the Object Removal " + al);
    }
}

```

Output

```

Initial ArrayList [Geeks, For, Geeks]
After the Index Removal [Geeks, Geeks]
After the Object Removal [Geeks]

```

5. Accessing Elements

To access an element in the list, we can use the [get\(\)](#) method, which returns the element at the specified index.

get(int index): This method returns the element at the specified index in the list.



```

import java.util.*;

class Geeks {

    public static void main(String args[])
    {

        List<String> al = new ArrayList<>();

        // Adding elements to object of List interface
        al.add("Geeks");
        al.add("For");
        al.add("Geeks");

        // Accessing elements using get() method
        String first = al.get(0);
        String second = al.get(1);
        String third = al.get(2);
    }
}

```

```

        System.out.println(first);
        System.out.println(second);
        System.out.println(third);
        System.out.println(al);
    }
}

```

Output

```

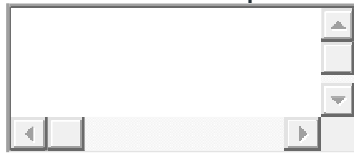
Geeks
For
Geeks
[Geeks, For, Geeks]

```

6. Checking if an element is present or not

To check if an element is present in the list, we can use the [contains\(\)](#) method. This method returns true if the specified element is present in the list, otherwise, it returns false.

contains(Object o): This method takes a single parameter, the object to be checked if it is present in the list.



```

import java.util.*;

class Geeks {

    public static void main(String args[])
    {

        List<String> al = new ArrayList<>();

        // Adding elements to object of List interface
        al.add("Geeks");
        al.add("For");
        al.add("Geeks");

        // Checking if element is present using contains()
method    boolean isPresent = al.contains("Geeks");

```

```

        // Printing the result
        System.out.println("Is Geeks present in the list? "+
isPresent);
    }
}

```

Output

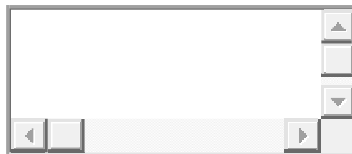
```
Is Geeks present in the list? true
```

Iterating over List Interface in Java

For larger datasets, Lists can be iterated using:

- Basic for loop with get(index)
- Enhanced for-each loop

These methods allow efficient traversal of elements.



```

import java.util.*;

public class Geeks {

    public static void main(String args[])
    {
        // Creating an empty ArrayList of string type
        List<String> al = new ArrayList<>();

        // Adding elements to above object of ArrayList
        al.add("Geeks");
        al.add("Geeks");

        // Adding element at specified position inside list
object    al.add(1, "For");

        // Using for loop for iteration
        for (int i = 0; i < al.size(); i++) {

            // Using get() method to access particular element
            System.out.print(al.get(i) + " ");

        }

        // New line for better readability
    }
}

```

```

        System.out.println();

        // Using for-each loop for iteration
        for (String str : al)

            System.out.print(str + " ");
    }
}

```

Output

```

Geeks For Geeks
Geeks For Geeks

```

Methods of the List Interface

Methods	Description
<u>add(int index, element)</u>	This method is used with Java List Interface to add an element at a particular index in the list. When a single parameter is passed, it simply adds the element at the end of the list.
<u>addAll(int index, Collection collection)</u> <u>size()</u>	This method is used with List Interface in Java to add all the elements in the given collection to the list. When a single parameter is passed, it adds all the elements of the given collection at the end of the list. This method is used with Java List Interface to return the size of the list.
<u>clear()</u>	This method is used to remove all the elements in the list. However, the reference of the list created is still stored.

Methods	Description
<u>remove(int index)</u>	This method removes an element from the specified index. It shifts subsequent elements(if any) to left and decreases their indexes by 1.
<u>remove(element)</u>	This method is used with Java List Interface to remove the first occurrence of the given element in the list.
<u>get(int index)</u>	This method returns elements at the specified index.
<u>set(int index, element)</u>	This method replaces elements at a given index with the new element. This function returns the element which was just replaced by a new element.
<u>indexOf(element)</u>	This method returns the first occurrence of the given element or -1 if the element is not present in the list.
<u>lastIndexOf(element)</u> <u>equals(element)</u>	This method returns the last occurrence of the given element or -1 if the element is not present in the list. This method is used with Java List Interface to compare the equality of the given element with the elements of the list.

Methods	Description
<u>hashCode()</u>	This method is used with List Interface in Java to return the hashcode value of the given list.
<u>isEmpty()</u>	This method is used with Java List Interface to check if the list is empty or not. It returns true if the list is empty, else false.
<u>contains(element)</u>	This method is used with List Interface in Java to check if the list contains the given element or not. It returns true if the list contains the element.
<u>containsAll(Collection collection)</u>	This method is used with Java List Interface to check if the list contains all the collection of elements.
sort(Comparator comp)	This method is used with List Interface in Java to sort the elements of the list on the basis of the given <u>comparator</u> .

List vs Set

Both the List interface and the Set interface inherits the Collection interface. However, there exists some differences between them.

List	Set
The List is an ordered sequence.	The Set is an unordered sequence.

List	Set
List allows duplicate elements	Set doesn't allow duplicate elements.
Elements by their position can be accessed.	Position access to elements is not allowed.
Multiple null elements can be stored.	The null element can store only once.
List implementations are ArrayList, LinkedList, Vector, Stack	Set implementations are HashSet, LinkedHashSet.

Stack Class in Java

Last Updated : 23 Mar, 2026

In Java, a Stack is a linear data structure that follows the Last In First Out (LIFO) principle and is defined in the [java.util](#) package. Internally, it extends the Vector class.

- Stack class maintains insertion order and allows duplicates and null values.
- Grows dynamically when its capacity is exceeded.
- All the methods of Stack are synchronized.
- It is thread-safe.
- Stack class implements List, RandomAccess, Cloneable, and Serializable interfaces.

The Stack class is a legacy class from early versions of Java. For new code, it is generally recommended to use [ArrayDeque](#) or [LinkedList](#) to implement stack behavior, as they offer better performance and flexibility in single-threaded scenarios.



```
import java.util.Stack;

public class Geeks {
    public static void main(String[] args)
    {
        // Create a new stack
        Stack<Integer> s = new Stack<>();

        // Push elements onto the stack
        s.push(1);
        s.push(2);
        s.push(3);
        s.push(4);

        // Pop elements from the stack
        while (!s.isEmpty()) {
            System.out.println(s.pop());
        }
    }
}
```

Output

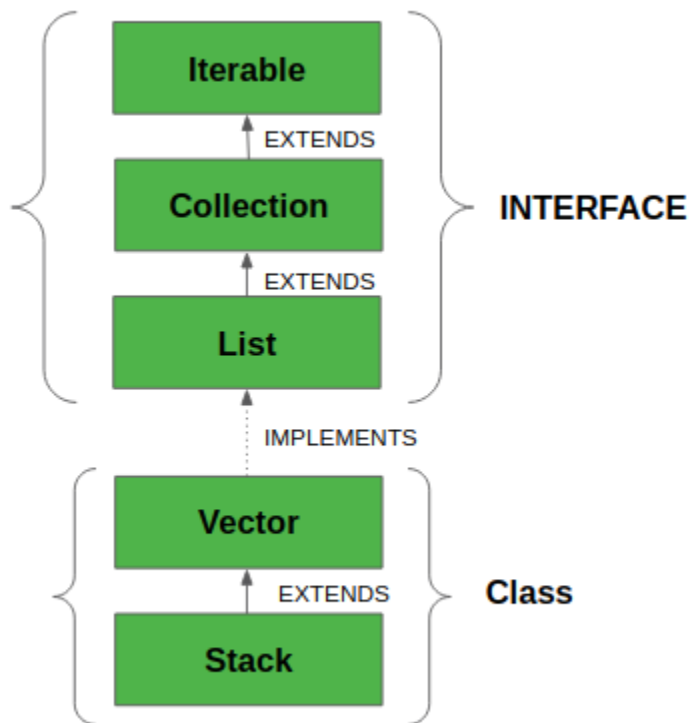
```
4
3
2
1
```

Explanation:

- Elements are added to the stack using the push() method, placing each item on top.
- A loop runs while the stack is not empty, checked using the isEmpty() method.
- Elements are removed and printed using the pop() method in LIFO order.

Hierarchy of Stack class

The below diagram shows the hierarchy of the Stack class:



Constructors of Stack in Java

In order to create a Stack, we need to create an object of the Stack class. The Stack class inherits constructors from the Vector class.

1. Stack()

This constructor is used to create an empty stack.

```
Stack<Integer> stack = new Stack<>()
```

2. Stack(int initialCapacity)

This constructor is used to create a stack with a specified initial capacity.

```
Stack<Integer> stack = new Stack<>(20);
```

Performing Different Operations on Stack Class

1. Adding Elements

With the help of [push\(\)](#) method we can add element to the stack. The push() method place the element at the top of the stack.



```
import java.io.*;
import java.util.*;

class Geeks {

    public static void main(String[] args)
    {
        // Default initialization of Stack
        Stack stack1 = new Stack();

        // Initialization of Stack using Generics
        Stack<String> stack2 = new Stack<String>();

        // pushing the elements
        stack1.push("4");
        stack1.push("All");
        stack1.push("Geeks");

        stack2.push("Geeks");
        stack2.push("For");
        stack2.push("Geeks");

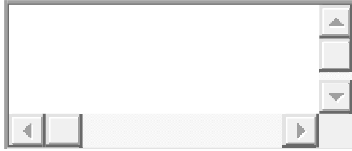
        // Printing the Stack Elements
        System.out.println(stack1);
        System.out.println(stack2);
    }
}
```

Output

```
[4, All, Geeks]
[Geeks, For, Geeks]
```

2. Accessing the Element

With the help of `peek()` method we can fetch the top element of the stack.



```
import java.io.*;
import java.util.*;

public class Geeks {

    public static void main(String args[])
    {
        // Creating an empty Stack
        Stack<String> stack = new Stack<String>();

        // Use push() to add elements into the Stack
        stack.push("Welcome");
        stack.push("To");
        stack.push("Geeks");
        stack.push("For");
        stack.push("Geeks");

        // Displaying the Stack
        System.out.println("Initial Stack: " + stack);

        // Fetching the element at the head of the Stack
        System.out.println("The element at the top of the"
                           + " stack is: " + stack.peek());

        System.out.println("Final Stack: " + stack);
    }
}
```

Output

```
Initial Stack: [Welcome, To, Geeks, For, Geeks]
The element at the top of the stack is: Geeks
Final Stack: [Welcome, To, Geeks, For, Geeks]
```

3. Removing Elements

With the help of [pop\(\)](#) method we can delete and return the top element from the stack.



```
import java.io.*;
import java.util.*;

public class Geeks {
    public static void main(String args[])
    {
        // Creating an empty Stack
        Stack<Integer> stack = new Stack<Integer>();

        // Use add() method to add elements
        stack.push(10);
        stack.push(15);
        stack.push(30);
        stack.push(20);
        stack.push(5);

        System.out.println("Initial Stack: " + stack);

        // Removing elements using pop() method
        System.out.println("Popped element: "
                           + stack.pop());
        System.out.println("Popped element: "
                           + stack.pop());

        // Displaying the Stack after pop operation
        System.out.println("Stack after pop operation "
                           + stack);

        System.out.println("Is stack empty? "
                           + stack.empty());

        // Pop remaining elements
        stack.pop();
        stack.pop();
        stack.pop();

        // Check if the stack is empty
        System.out.println("Is stack empty? "
                           + stack.empty());
    }
}
```

```
}  
}
```

Output

```
Initial Stack: [10, 15, 30, 20, 5]  
Popped element: 5  
Popped element: 20  
Stack after pop operation [10, 15, 30]  
Is stack empty? false  
Is stack empty? True
```

Methods Inherited from the Vector Class

Method	Description
<u>add(Object obj)</u>	This method is used to appends the specified element to the end of the Vector.
<u>add(int index, Object obj)</u>	This method is used to inserts the specified element at the specified position in the Vector.
<u>addAll(Collection c)</u>	This method is used to appends all of the elements in the specified Collection to the end of theVector, in the order that they are returned by the specified Collection's Iterator.
<u>addAll(int index, Collection c)</u>	This method is used to inserts all the elements in the specified Collection into this Vector at the specified position.

Method	Description
<u>addElement(Object o)</u>	This method is used to adds the specified component to the end of the vector
<u>capacity()</u>	This method returns the current capacity of the vector.
<u>clear()</u>	This method removes all the elements from the Vector.
<u>clone()</u>	This method returns a clone of the vector.
<u>contains(Object o)</u>	This method returns true if this vector contains the specified element.
<u>containsAll(Collection c)</u>	This method returns true if this Vector contains all the elements in the specified Collection.
<u>copyInto(Object []array)</u>	This method is used to copies the components of this vector into the specified array.
<u>elementAt(int index)</u>	This method returns the component at the specified index.
<u>elements()</u>	This method returns an enumeration of the components of the vector.

Method	Description
<u>ensureCapacity(int minCapacity)</u>	This method increases the capacity of the vector, if necessary, to ensure that it can hold at least the number of components specified by the minimum capacity argument.
<u>equals()</u>	This method compares the specified Object with the Vector for equality.
<u>firstElement()</u>	This method returns the first component (the item at index 0) of the vector.
<u>get(int index)</u>	This method returns the element at the specified position in the Vector.
<u>hashCode()</u>	This method returns the hash code value for the Vector.
<u>indexOf(Object o)</u>	This method returns the index of the first occurrence of the specified element in this vector or -1 if this vector does not contain the element.
<u>indexOf(Object o, int index)</u>	This method returns the index of the first occurrence of the specified element in this vector, searching forwards from the index or returns -1 if the element is not found.

Method	Description
<u>insertElementAt(Object o, int index)</u>	This method inserts the specified object as a component in this vector at the specified index.
<u>isEmpty()</u>	This method tests if this vector has no components.
<u>iterator()</u>	This method returns an iterator over the elements in this list in proper sequence.
<u>lastElement()</u>	This method returns the last component of the vector.
<u>lastIndexOf(Object o)</u>	This method returns the index of the last occurrence of the specified element in this vector or -1 If this vector does not contain the element.
<u>lastIndexOf(Object o, int index)</u>	This method returns the index of the last occurrence of the specified element in this vector, searching backward from the index or returns -1 if the element is not found.
<u>listIterator()</u>	This method returns a list iterator over the elements in this list (in proper sequence).
<u>listIterator(int index)</u>	This method returns a list iterator over the elements in this list (in proper sequence), starting at the specified position in the list.

Method	Description
<u>remove(int index)</u>	This method removes the element at the specified position in this Vector.
<u>remove(Object o)</u>	This method removes the first occurrence of the specified element in this Vector. If the Vector does not contain the element, it is unchanged.
<u>removeAll(Collection c)</u>	This method removes from this Vector all of its elements that are contained in the specified Collection.
<u>removeAllElements()</u>	This method removes all components from this vector and sets its size to zero.
<u>removeElement(Object o)</u>	This method removes the first (lowest-indexed) occurrence of the argument from this vector.
<u>removeElementAt(int index)</u>	This method deletes the component at the specified index.
<u>removeRange(int fromIndex, int toIndex)</u>	This method removes from this list all the elements whose index is between fromIndex, inclusive, and toIndex, exclusive.
<u>retainAll(Collection c)</u>	This method retains only the elements in this Vector that are contained in the specified Collection.

Method	Description
<u>set(int index, Object o)</u>	This method replaces the element at the specified position in this Vector with the specified element.
<u>setElementAt(Object o, int index)</u>	This method sets the component at the specified index of this vector to be the specified object.
<u>setSize(int newSize)</u>	This method sets the size of this vector.
<u>size()</u>	This method returns the number of components in this vector.
<u>subList(int fromIndex, int toIndex)</u>	This method returns a view of the portion of this List between fromIndex, inclusive, and toIndex, exclusive.
<u>toArray()</u>	This method returns an array containing all of the elements in this Vector in the correct order.
<u>toArray(Object []array)</u>	This method returns an array containing all of the elements in this Vector in the correct order; the runtime type of the returned array is that of the specified array.
<u>toString()</u>	This method returns a string representation of this Vector, containing the String representation of each element.

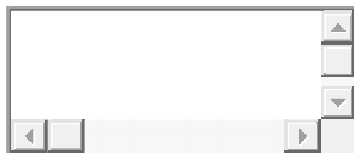
Method	Description
<u>trimToSize()</u>	This method trims the capacity of this vector to be the vector's current size.

Set in Java

Last Updated : 18 Nov, 2025

In Java, the Set interface is a part of the Java Collection Framework, located in the java.util package. It represents a collection of unique elements, meaning it does not allow duplicate values.

- The set interface does not allow duplicate elements.
- It can contain at most one null value except TreeSet implementation which does not allow null.
- The set interface provides efficient search, insertion, and deletion operations.



```
import java.util.HashSet;
import java.util.Set;

public class Geeks {

    public static void main(String args[])
    {
        // Create a Set using HashSet
        Set<String> s = new HashSet<>();

        // Displaying the Set
        System.out.println("Set Elements: " + s);
    }
}
```

Explanation: In the above example, HashSet will appear as an empty set, as no elements were added. The order of elements in HashSet is not

guaranteed, so the elements will be displayed in a random order if any are added.

Hierarchy of Java Set interface

The image below demonstrates the hierarchy of Java Set interface.

Set-Hierarchy

Classes that implement the Set interface

- **HashSet**: A set that stores unique elements without any specific order, using a hash table and allows one null element.
- **EnumSet**: A high-performance set designed specifically for enum types, where all elements must belong to the same enum.
- **LinkedHashSet**: A set that maintains the order of insertion while storing unique elements.
- **TreeSet**: A set that stores unique elements in sorted order, either by natural ordering or a specified comparator.

Declaration of Set Interface

The declaration of Set interface is listed below:

```
public interface Set extends Collection
```

Creating Set Objects

Since Set is an [interface](#), objects cannot be created of the typeset. We always need a class that implements this interface in order to create an object. And also, after the introduction of [Generics](#) in Java 1.5, it is possible to restrict the

type of object that can be stored in the Set. This type-safe set can be defined as:

```
// Obj is the type of the object to be stored in Set
Set<Obj> set = new HashSet<Obj> ();
```

Performing Various Operations on Set

Set interface provides commonly used operations to manage unique elements in a collection. Now let us discuss these operations individually as follows:

1. Adding Elements

To add elements to a Set in Java, use the `add()` method.



```
import java.util.*;

// Main class
class Geeks {

    public static void main(String[] args)
    {
        Set<String> s = new HashSet<String>();

        s.add("B");
        s.add("B");
        s.add("C");
        s.add("A");

        System.out.println(s);
    }
}
```

Output

```
[A, B, C]
```

2. Accessing the Elements

After adding the elements, if we wish to access the elements, we can use inbuilt methods like [contains\(\)](#).



```
import java.util.*;

class Geeks {

    public static void main(String[] args)
    {
        Set<String> h = new HashSet<String>();

        h.add("A");
        h.add("B");
        h.add("C");
        h.add("A");

        System.out.println("Set is " + h);

        String s = "D";

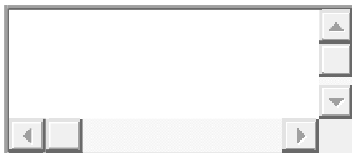
        System.out.println("Contains " + s + " " + h.contains(s));
    }
}
```

Output

```
Set is [A, B, C]
Contains D false
```

3. Removing Elements

The values can be removed from the Set using the [remove\(\) method](#).



```
import java.util.*;

class Geeks {

    public static void main(String[] args)
    {
        // Declaring object of Set of type String
        Set<String> h = new HashSet<String>();
```

```

        // Elements are added using add() method, Custom input
elements
        h.add("A");
        h.add("B");
        h.add("C");
        h.add("B");
        h.add("D");
        h.add("E");

        System.out.println("Initial HashSet " + h);

        // Removing custom element using remove() method
        h.remove("B");
        System.out.println("After removing element " + h);
    }
}

```

Output

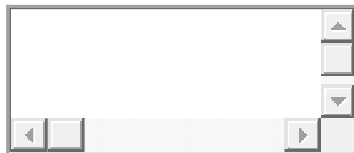
```

Initial HashSet [A, B, C, D, E]
After removing element [A, C, D, E]

```

4. Iterating elements

There are various ways to iterate through the Set. The most famous one is to use the enhanced for loop.



```

import java.util.*;

class Geeks {

    public static void main(String[] args)
    {
        // Creating object of Set and declaring String type
        Set<String> h = new HashSet<String>();

        // Adding elements to Set using add() method, Custom input
elements
        h.add("A");
        h.add("B");
    }
}

```



```
h.add("C");
h.add("B");
h.add("D");
h.add("E");

// Iterating through the Set via for-each loop
for (String value : h)

    // Printing all the values inside the object
    System.out.print(value + ", ");

System.out.println();
}
```

[Try it on GfG Practice](#)

Output

A, B, C, D, E,

Methods of Set Interface

Let us discuss methods present in the Set interface provided below in a tabular format below as follows:

Method	Description
<u>add(element)</u>	Adds element if not already present. Returns true if added.
<u>addAll(collection)</u>	Adds all elements from the given collection.

Method	Description
<u>clear()</u>	Removes all elements from the set.
<u>contains(element)</u>	Checks if the set contains the specified element.
<u>containsAll(collection)</u>	Checks if the set contains all elements from the given collection.
<u>hashCode()</u>	Returns the hash code of the set.
<u>isEmpty()</u>	This method is used to check whether the set is empty or not.
<u>iterator()</u>	This method is used to return the <u>iterator</u> of the set.
<u>remove(element)</u>	Removes the specified element from the set.
<u>removeAll(collection)</u>	Removes all elements in the given collection from the set.
<u>retainAll(collection)</u>	Retains only elements present in the given collection.
<u>size()</u>	Returns the number of elements in the set.

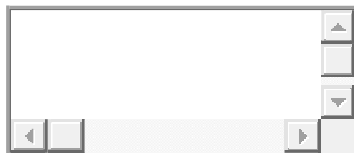
Method	Description
toArray()	This method is used to form an array of the same elements as that of the Set.

TreeSet in Java

Last Updated : 22 Nov, 2025

A TreeSet is a collection class that stores unique elements in a sorted order. It is part of [java.util package](#) that implements the SortedSet interface, and internally uses a Red-Black tree to maintain sorting.

- Does not allow duplicates and null values. From JDK 7 onward, inserting null throws NullPointerException.
- Implements the [NavigableSet interface](#) and provides navigation methods like higher(), lower(), ceiling() and floor().
- TreeSet is not synchronized. it must be synchronized using [Collections.synchronizedSet\(\)](#).



```
import java.util.TreeSet;

public class TreeSetCreation
{
    public static void main(String args[])
    {
        // Create a TreeSet of Strings
        TreeSet<String> t = new TreeSet<>();

        // Displaying the TreeSet (which is empty at this point)
        System.out.println("TreeSet elements: " + t);
    }
}
```

Output

```
TreeSet elements: []
```

Hierarchy Diagram of TreeSet

TreeSet implements [NavigableSet](#), which extends [SortedSet](#), which extends [Set](#).

TreeSet-Hierarchy

Note:

- A class must implement [Comparable](#) (or provide a *Comparator*) to be stored in a *TreeSet*.
- Built-in classes like *String*, wrapper classes, etc., already implement *Comparable*

Constructors of TreeSet

In order to create a *TreeSet*, we need to create an object of the *TreeSet* class. The *TreeSet* class consists of various constructors which allow the possible creation of the *TreeSet*. The following are the constructors available in this class:

1. TreeSet()

Creates an empty *TreeSet* that sorts elements in their natural order

```
TreeSet ts = new TreeSet();
```

2. TreeSet(Comparator)

This constructor is used to build an empty *TreeSet* object in which elements will need an external specification of the sorting order.

```
TreeSet ts = new TreeSet(Comparator comp);
```

3. TreeSet(Collection)

Creates a TreeSet containing all elements from the given collection, stored in their natural sorted order. It is useful for converting a collection to a TreeSet.

```
TreeSet t = new TreeSet(Collection col);
```

4. TreeSet(SortedSet)

Creates a TreeSet containing the same elements and order as the specified SortedSet.

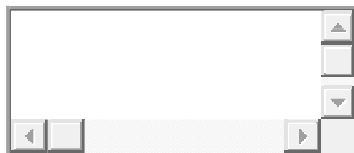
```
TreeSet t = new TreeSet(SortedSet s);
```

Common Operations on TreeSet

Here we will be performing various operations over the TreeSet object to get familiar with the methods and concepts of TreeSet in java. Let's see how to perform a few frequently used operations on the TreeSet.

1. Adding Elements

To add elements to a TreeSet, use the add() method. Elements are stored in ascending order, duplicates are ignored, and null values are not allowed.



```
import java.util.*;
class GFG {

    public static void main(String[] args)
    {
        // Creating a Set interface with reference to TreeSet class
        Set<String> ts = new TreeSet<>();

        // Elements are added using add() method
        ts.add("Geek");
        ts.add("For");
        ts.add("Geeks");

        // Print all elements inside object
        System.out.println(ts);
    }
}
```

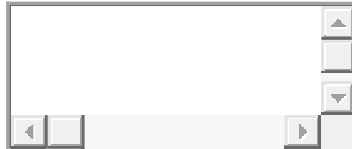
```
}
```

Output

```
[For, Geek, Geeks]
```

2. Accessing the Elements

After adding the elements, if we wish to access the elements, we can use inbuilt methods like [contains\(\)](#), [first\(\)](#), [last\(\)](#), etc.



```
import java.util.*;
class GFG {

    public static void main(String[] args){

        // Creating a NavigableSet object with reference to
        // TreeSet class
        NavigableSet<String> ts = new TreeSet<>();

        // Elements are added using add() method
        ts.add("Geek");
        ts.add("For");
        ts.add("Geeks");

        System.out.println("Tree Set is " + ts);
        String check = "Geeks";

        // Check if the above string exists in the treeset
        // or not
        System.out.println("Contains " + check + " "
                           + ts.contains(check));

        // Print the first element in the TreeSet
        System.out.println("First Value " + ts.first());

        // Print the last element in the TreeSet
        System.out.println("Last Value " + ts.last());

        String val = "Geek";

        // Find the values just greater and smaller than the
```

```

        // above string
        System.out.println("Higher " + ts.higher(val));
        System.out.println("Lower " + ts.lower(val));
    }
}

```

Output

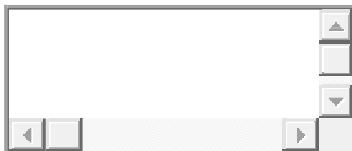
```

Tree Set is [For, Geek, Geeks]
Contains Geeks true
First Value For
Last Value Geeks
Higher Geeks
Lower For

```

3. Removing the Values

The values can be removed from the TreeSet using the [remove\(\)](#) method. There are various other methods that are used to remove the first value or the last value.



```

import java.util.*;

class GFG {

    public static void main(String[] args){

        // Creating an object of NavigableSet with reference
        // to TreeSet class
        NavigableSet<String> ts = new TreeSet<>();

        // Elements are added using add() method
        ts.add("Geek");
        ts.add("For");
        ts.add("Geeks");
        ts.add("A");
        ts.add("B");
        ts.add("Z");
    }
}

```

```

        System.out.println("Initial TreeSet " + ts);

        // Removing a specific existing element inserted
        // above
        ts.remove("B");

        // Printing the updated TreeSet
        System.out.println("After removing element " + ts);

        // Now removing the first element using pollFirst()
        // method
        ts.pollFirst();

        // Again printing the updated TreeSet
        System.out.println("After removing first " + ts);

        // Removing the last element using pollLast() method
        ts.pollLast();

        System.out.println("After removing last " + ts);
    }
}

```

Output

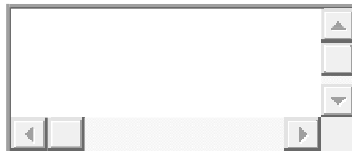
```

Initial TreeSet [A, B, For, Geek, Geeks, Z]
After removing element [A, For, Geek, Geeks, Z]
After removing first [For, Geek, Geeks, Z]
After removing last [For, Geek, Geeks]

```

4. Iterating the TreeSet

There are various ways to iterate through the TreeSet. The most famous one is to use the [enhanced for loop](#).



```

import java.util.*;

class GFG{

    public static void main(String[] args){

```



```

// Creating an object of Set with reference to
// TreeSet class
Set<String> ts = new TreeSet<>();

// Adding elements in above object using add()
// method
ts.add("Geek");
ts.add("For");
ts.add("Geeks");
ts.add("A");
ts.add("B");
ts.add("Z");

for (String value : ts)
    System.out.print(value + ", ");

System.out.println();
}
}

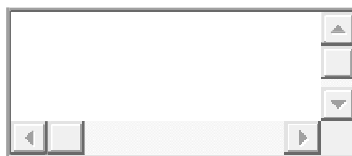
```

Output

```
A, B, For, Geek, Geeks, Z,
```

5. Using TreeSet with Custom Objects

If a class does not implement Comparable (e.g., StringBuffer), provide a Comparator.



```

import java.util.*;

class Geeks {

    public static void main(String[] args)
    {
        // Creating a TreeSet with a custom Comparator
        Set<StringBuffer> ts = new TreeSet<>(new
        Comparator<StringBuffer>() {

```

```

        @Override
        public int compare(StringBuffer sb1, StringBuffer sb2) {
            return sb1.toString().compareTo(sb2.toString());
        }
    });

    // Adding elements to the TreeSet
    ts.add(new StringBuffer("A"));
    ts.add(new StringBuffer("Z"));
    ts.add(new StringBuffer("L"));
    ts.add(new StringBuffer("B"));
    ts.add(new StringBuffer("O"));
    ts.add(new StringBuffer("1"));

    // Printing the elements
    System.out.println(ts);
}

```

Try it on GfG Practice

Output

```
[1, A, B, L, O, Z]
```

Methods of TreeSet

Method of TreeSet Class are depicted below in tabular format which later on we will be implementing to showcase in the implementation part.

Method	Description
<u>add(Object o)</u>	Adds element in sorted order; ignores duplicates.

Method	Description
<u>addAll(Collection c)</u>	Adds all elements from a collection; ignores duplicates..
<u>ceiling?(E e)</u>	This method returns the least element in this set greater than or equal to the given element, or null if there is no such element.
<u>clear()</u>	This method will remove all the elements.
<u>clone()</u>	The method is used to return a shallow copy of the set, which is just a simple copied set.
<u>Comparator comparator()</u>	This method will return the Comparator used to sort elements in TreeSet or it will return null if the default natural sorting order is used.
<u>contains(Object o)</u>	This method will return true if a given element is present in TreeSet else it will return false.
<u>descendingIterator?()</u>	This method returns an iterator over the elements in this set in descending order.
<u>descendingSet?()</u>	This method returns a reverse order view of the elements contained in this set.

Method	Description
<u>first()</u>	This method will return the first element in TreeSet if TreeSet is not null else it will throw NoSuchElementException.
<u>floor?(E e)</u>	This method returns the greatest element in this set less than or equal to the given element, or null if there is no such element.
<u>headSet(Object toElement)</u>	This method will return elements of TreeSet which are less than the specified element.
<u>higher?(E e)</u>	This method returns the least element in this set strictly greater than the given element, or null if there is no such element.
<u>isEmpty()</u>	This method is used to return true if this set contains no elements or is empty and false for the opposite case.
<u>Iterator iterator()</u>	Returns an iterator for iterating over the elements of the set.
<u>last()</u>	This method will return the last element in TreeSet if TreeSet is not null else it will throw NoSuchElementException.

Method	Description
<u>lower?(E e)</u>	This method returns the greatest element in this set strictly less than the given element, or null if there is no such element.
<u>pollFirst?()</u>	This method retrieves and removes the first (lowest) element, or returns null if this set is empty.
<u>pollLast?()</u>	This method retrieves and removes the last (highest) element, or returns null if this set is empty.
<u>remove(Object o)</u>	It returns true/false whether the element was removed
<u>size()</u>	This method is used to return the size of the set or the number of elements present in the set.
<u>spliterator()</u>	This method creates a late-binding and fail-fast Spliterator over the elements in this set.
<u>subSet(Object fromElement, Object toElement)</u>	This method will return elements ranging from fromElement to toElement. fromElement is inclusive and toElement is exclusive.

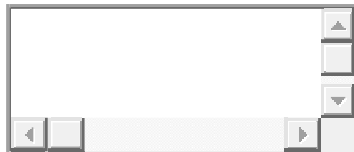
Method	Description
<u>tailSet(Object fromElement)</u>	This method will return elements of TreeSet which are greater than or equal to the specified element.

Java Comparable Interface

Last Updated : 2 Feb, 2026

The Comparable interface in Java is used to define the natural ordering of objects of a class. It enables objects to be compared and sorted automatically without using an external Comparator.

- It contains the `compareTo()` method, which compares the current object with another object.
- It is commonly used with [Collections.sort\(\)](#) and [Arrays.sort\(\)](#) for sorting custom objects.



```
import java.util.*;

class Student implements Comparable<Student> {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public int compareTo(Student other) {
        return this.id - other.id; // Sort by ID in ascending order
    }

    @Override
```

```

        public String toString() {
            return id + "-" + name;
        }
    }

    public class ComparableExample {
        public static void main(String[] args) {
            List<Student> students = new ArrayList<>();
            students.add(new Student(3, "Alice"));
            students.add(new Student(1, "Bob"));
            students.add(new Student(2, "Charlie"));

            Collections.sort(students); // Sorts using compareTo()

            System.out.println(students);
        }
    }

```

Output

```
[1-Bob, 2-Charlie, 3-Alice]
```

Explanation

- Student implements Comparable<Student> and overrides compareTo() to define the sorting logic.
- compareTo() returns a negative, zero, or positive value to indicate order.
- Collections.sort(students) sorts the list automatically using the natural order defined in compareTo().
- Output shows the students sorted by ID in ascending order.

Syntax

```

class MyClass implements Comparable<MyClass> {
    @Override
    public int compareTo(MyClass obj) {
        // comparison logic
    }
}

```

- Returns negative if this < obj.

- Returns 0 if this == obj.
- Returns positive if this > obj.

Key Method: `int compareTo(T obj)` - Compares the current object with the specified object.

Example: Sorting by Marks



```
import java.util.*;

class Student implements Comparable<Student> {
    String name;
    int marks;

    Student(String name, int marks) {
        this.name = name;
        this.marks = marks;
    }

    @Override
    public int compareTo(Student other) {
        return this.marks - other.marks; // ascending order by marks
    }

    @Override
    public String toString() {
        return name + ": " + marks;
    }
}

public class ComparableExample {
    public static void main(String[] args) {
        List<Student> students = new ArrayList<>();
        students.add(new Student("Alice", 85));
        students.add(new Student("Bob", 92));
        students.add(new Student("Charlie", 78));

        Collections.sort(students);

        for (Student s : students) {
            System.out.println(s);
        }
    }
}
```



```
}  
}
```

Output

```
Charlie: 78
```

```
Alice: 85
```

```
Bob: 92
```

Explanation:

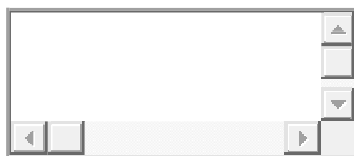
- Defines natural order by marks using compareTo().
- Collections.sort() automatically arranges elements in ascending order.
- The toString() method is used to print objects in a readable format.

Queue Interface In Java

Last Updated : 24 Mar, 2026

The Queue interface is part of the [java.util](#) package and extends the [Collection interface](#). It represents a data structure where elements are processed based on a specific order.

- Elements follow FIFO (First-In-First-Out) in LinkedList and priority order in PriorityQueue
- Elements cannot be accessed directly using an index
- Allows storing duplicate elements



```
import java.util.PriorityQueue;  
import java.util.Queue;  
  
public class Geeks{  
  
    public static void main(String[] args){  
  
        // Create a PriorityQueue of Integers  
        Queue<Integer> pq = new PriorityQueue<>();
```

```

    // Adding elements to the PriorityQueue
    pq.add(50);
    pq.add(20);
    pq.add(40);
    pq.add(10);
    pq.add(30);

    // Display the PriorityQueue elements
    System.out.println("PriorityQueue elements: " + pq);
}
}

```

Output

```
PriorityQueue elements: [10, 20, 40, 50, 30]
```

Note: [PriorityQueue](#) arranges elements according to priority order (ascending by default), not insertion order.

Declaration of Java Queue Interface

public interface Queue extends Collection

We cannot instantiate a Queue directly as it is an interface. Here, we can use a PriorityQueue class that implements this interface.

```
Queue<Obj> queue = new LinkedList<Obj>();
```

Hierarchy of Queue Interface

It extends the [Collection interface](#) and has implementations like [LinkedList](#), [ArrayDeque](#), [PriorityQueue](#).

Common Implementations of Queue Interface

- [LinkedList](#): Implements List and Deque interfaces, allows null elements, and can be used as a FIFO queue when used through the Queue interface.
- [ArrayDeque](#): A resizable array-based queue that is faster than LinkedList and does not allow nulls.
- [PriorityQueue](#): A queue where elements are processed according to their priority instead of insertion order.

- [ConcurrentLinkedQueue](#): A thread-safe, non-blocking queue suitable for concurrent environments.
- [BlockingQueue](#): A thread-safe queue that supports blocking operations for producer-consumer scenarios.

Different Operations on Queue Interface using PriorityQueue class

1. Adding Elements

To add an element in a queue, we can use the [add\(\) method](#). The insertion order is not retained in the PriorityQueue. The elements are stored based on the priority order which is ascending by default.



```
import java.util.*;

public class Geeks {

    public static void main(String args[])
    {
        Queue<String> pq = new PriorityQueue<>();

        pq.add("Geeks");
        pq.add("For");
        pq.add("Geeks");

        System.out.println(pq);
    }
}
```

Output

```
[For, Geeks, Geeks]
```

2. Removing Elements

To remove an element from a queue, we can use the [remove\(\) method](#). If there are multiple objects, then the first occurrence of the object is removed. The **poll() method** is also used to remove the head and return it.



```
import java.util.*;

public class Geeks {

    public static void main(String args[])
    {
        Queue<String> pq = new PriorityQueue<>();

        pq.add("Geeks");
        pq.add("For");
        pq.add("Geeks");

        System.out.println("Initial Queue: " + pq);

        pq.remove("Geeks");

        System.out.println("After Remove: " + pq);

        System.out.println("Poll Method: " + pq.poll());

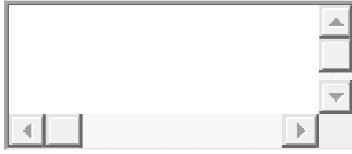
        System.out.println("Final Queue: " + pq);
    }
}
```

Output

```
Initial Queue: [For, Geeks, Geeks]
After Remove: [For, Geeks]
Poll Method: For
Final Queue: [Geeks]
```

3. Accessing Elements

We can access the head element without removing it using `peek()` or `element()`.



```
import java.util.PriorityQueue;
import java.util.Queue;

public class Geeks{

    public static void main(String[] args) {
        // Create a PriorityQueue of Strings
        Queue<String> pq = new PriorityQueue<>();

        // Adding elements to the queue
        pq.add("Geeks");
        pq.add("For");
        pq.add("Geeks");

        // Access the head element without removing
        System.out.println("Head using peek(): " + pq.peek());
        System.out.println("Head using element(): " + pq.element());

        // Display the queue to show elements are not removed
        System.out.println("Queue after accessing head: " + pq);
    }
}
```

Output

```
Head using peek(): For
Head using element(): For
Queue after accessing head: [For, Geeks, Geeks]
```

4. Iterating the Queue

There are multiple ways to iterate through the Queue. The most famous way is converting the queue to the array and traversing using the **for loop**. The queue has also an inbuilt iterator which can be used to iterate through the queue.



```
import java.util.*;

public class Geeks {

    public static void main(String args[])
    {
        Queue<String> pq = new PriorityQueue<>();

        pq.add("Geeks");
        pq.add("For");
        pq.add("Geeks");
        // use Type safe Iterator
        Iterator<String> iterator = pq.iterator();

        while (iterator.hasNext()) {
            System.out.print(iterator.next() + " ");
        }
    }
}
```

Output

For Geeks Geeks

Methods of Queue Interface

Method	Description
<u>add(E e)</u>	Inserts the specified element; throws exception if insertion fails.
<u>offer(E e)</u>	Inserts the specified element; returns false if insertion fails.

Method	Description
<u>remove()</u>	Removes and returns the head of the queue; throws exception if empty.
<u>poll()</u>	Removes and returns the head; returns null if empty.
<u>peek()</u>	Retrieves, but does not remove, the head; returns null if empty.
<u>size()</u>	Returns the number of elements in the queue.
<u>isEmpty()</u>	Returns true if the queue contains no elements.
<u>contains(Object o)</u>	Returns true if the queue contains the specified element.
<u>iterator()</u>	Returns an iterator over the elements in the queue.
<u>toArray()</u>	Converts the queue elements into an array.
<u>addFirst(E e)</u>	Inserts element at the front (Deque only).
<u>addLast(E e)</u>	Inserts element at the end (Deque only).
<u>offerFirst(E e)</u>	Inserts element at the front; returns false if fails (Deque only).

Method	Description
<u>offerLast(E e)</u>	Inserts element at the end; returns false if fails (Deque only).
<u>removeFirst()</u>	Removes and returns the first element (Deque only).
<u>removeLast()</u>	Removes and returns the last element (Deque only).
<u>pollFirst()</u>	Removes and returns the first element; returns null if empty (Deque only).
<u>pollLast()</u>	Removes and returns the last element; returns null if empty (Deque only).
<u>getFirst()</u>	Retrieves, but does not remove, the first element (Deque only).
<u>getLast()</u>	Retrieves, but does not remove, the last element (Deque only).
<u>peekFirst()</u>	Retrieves, but does not remove, the first element; returns null if empty (Deque only).
<u>peekLast()</u>	Retrieves, but does not remove, the last element; returns null if empty (Deque only).

Method	Description
<code>put(E e)</code>	Inserts element, waits if necessary (BlockingQueue only).
<code>take()</code>	Removes and returns head element, waits if empty (BlockingQueue only).

Map Interface in Java

Last Updated : 7 Jan, 2026

In Java, the Map Interface is part of the [java.util package](#) and represents a collection of key-value pairs, where Keys should be unique, but values can be duplicated.

- It provides efficient retrieval, insertion, and deletion operations based on keys.
- [HashMap](#) and [LinkedHashMap](#) allow one null key, and [TreeMap](#) does NOT allow null keys (if natural ordering is used).
- Use `ConcurrentHashMap` for thread-safe operations, or `Collections.synchronizedMap()` to make an existing map synchronized.

Declaration of the Map interface

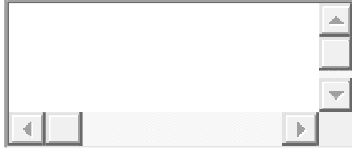
```
public interface Map<K, V>
```

- K -> Type of keys maintained by the map
- V -> Type of mapped values

Creating Map Objects

Since a Map is an interface, we cannot create its object directly. We must use a class that implements it (e.g., HashMap).

```
Map<String, Integer> hm = new HashMap<>();
```



```
import java.util.HashMap;
import java.util.Map;

public class Geeks{

    public static void main(String[] args){

        // Create a Map using HashMap
        Map<String, Integer> m = new HashMap<>();

        // Adding key-value pairs to the map
        m.put("Geek1", 1);
        m.put("Geek2", 2);
        m.put("Geek3", 3);

        System.out.println("Map elements: " + m);
    }
}
```

Output

```
Map elements: {Geek3=3, Geek2=2, Geek1=1}
```

Hierarchy of Map

It is part of the Java Collections Framework, and its key implementation classes include [HashMap](#), [LinkedHashMap](#), [TreeMap](#), and [Hashtable](#).

Implemented Classes of Map Interface

- [HashMap](#): Stores key-value pairs using hashing for fast access, insertion, and deletion.
- [LinkedHashMap](#): Similar to HashMap but maintains the insertion order of key-value pairs.

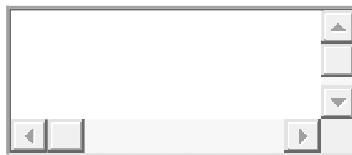
- **TreeMap:** Stores key-value pairs in sorted order using natural ordering or a custom comparator.
- **Hashtable:** A synchronized Map implementation that doesn't allow null keys or values.

Operations on Map using HashMap

Now, let's see how to perform a few frequently used operations on a Map using the widely used HashMap class.

1. Adding Elements

Use the `put()` method to add elements to a Map. In HashMap, insertion order isn't preserved, each element is stored based on its hash for faster access.



```
import java.util.*;

class Geeks {

    public static void main(String[] args) {

        // Default initialization of a Map using generics
        Map<Integer, String> hm1 = new HashMap<>();

        // Initialization of a Map (redundant type on right side
        // unnecessary)
        Map<Integer, String> hm2 = new HashMap<>();

        // Inserting elements
        hm1.put(1, "Geeks");
        hm1.put(2, "For");
        hm1.put(3, "Geeks");

        // Inserting elements – no need to use new Integer()
        hm2.put(1, "Geeks");
        hm2.put(2, "For");
        hm2.put(3, "Geeks");
    }
}
```

```

        System.out.println(hm1);
        System.out.println(hm2);
    }
}

```

Output

```

{1=Geeks, 2=For, 3=Geeks}
{1=Geeks, 2=For, 3=Geeks}

```

2. Changing Element

To update a value, use the [put\(\)](#) method with the same key. The new value replaces the old one for that key.



```

import java.util.*;

class Geeks {
    public static void main(String args[])
    {

        // Initialization of a Map using Generics
        Map<Integer, String> hm1
            = new HashMap<Integer, String>();

        // Inserting the Elements
        hm1.put(new Integer(1), "Geeks");
        hm1.put(new Integer(2), "Geeks");
        hm1.put(new Integer(3), "Geeks");

        System.out.println("Initial Map: " + hm1);

        hm1.put(new Integer(2), "For");

        System.out.println("Updated Map: " + hm1);
    }
}

```

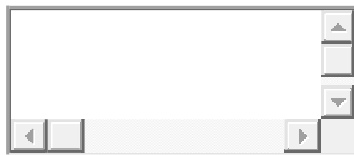
Output

Initial Map: {1=Geeks, 2=Geeks, 3=Geeks}

Updated Map: {1=Geeks, 2=For, 3=Geeks}

3. Removing Elements

To remove an element from the Map, we can use the `remove()` method. This method takes the key value and removes the mapping for a key from this map if it is present in the map.



```
import java.util.*;

class Geeks{

    public static void main(String args[]){

        // Initialization of a Map using Generics
        Map<Integer, String> hm1
            = new HashMap<Integer, String>();

        // Inserting the Elements
        hm1.put(new Integer(1), "Geeks");
        hm1.put(new Integer(2), "For");
        hm1.put(new Integer(3), "Geeks");
        hm1.put(new Integer(4), "For");

        System.out.println(hm1);

        hm1.remove(new Integer(4));

        System.out.println(hm1);
    }
}
```

Output

{1=Geeks, 2=For, 3=Geeks, 4=For}

{1=Geeks, 2=For, 3=Geeks}

4. Iterating through the Map

There are multiple ways to iterate through the Map. The most famous way is to use a for-each loop and get the keys. The value of the key is found by using the `getValue()` method.



```
import java.util.*;

class Geeks{
    public static void main(String args[]){

        // Initialization of a Map using Generics
        Map<Integer, String> hm1
            = new HashMap<Integer, String>();

        // Inserting the Elements
        hm1.put(new Integer(1), "Geeks");
        hm1.put(new Integer(2), "For");
        hm1.put(new Integer(3), "Geeks");

        for (Map.Entry mapElement : hm1.entrySet()) {
            int key = (int)mapElement.getKey();

            // Finding the value
            String value = (String)mapElement.getValue();

            System.out.println(key + " : " + value);
        }
    }
}
```

Output

```
1 : Geeks
2 : For
3 : Geeks
```

Methods in Java Map Interface

Methods	Action Performed
<u>clear()</u>	This method is used in Java Map Interface to clear and remove all of the elements or mappings from a specified Map collection.
<u>containsKey(Object)</u>	Checks if a key exists in the map.
<u>containsValue(Object)</u>	Checks if a value exists in the map.
<u>entrySet()</u>	Returns a set view of the map's key-value pairs.
<u>equals(Object)</u>	Compares two maps for equality.
<u>get(Object)</u>	Returns the value for the given key, or null if not found.
<u>hashCode()</u>	This method is used in Map Interface to generate a hashCode for the given map containing keys and values.
<u>isEmpty()</u>	This method is used to check if a map is having any entry for key and value pairs. If no mapping exists, then this returns true.
<u>keySet()</u>	Returns a set view of the keys in the map.

Methods	Action Performed
<u>put(Object, Object)</u>	This method is used in Java Map Interface to associate the specified value with the specified key in this map.
<u>putAll(Map)</u>	This method is used in Map Interface in Java to copy all of the mappings from the specified map to this map.
<u>remove(Object)</u>	This method is used in Map Interface to remove the mapping for a key from this map if it is present in the map.
<u>size()</u>	This method is used to return the number of key/value pairs available in the map.
<u>values()</u>	Returns a collection view of the map's values.
<u>getOrDefault(Object key, V defaultValue)</u>	Returns the value to which the specified key is mapped, or defaultValue if this map contains no mapping for the key.
<u>merge(K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)</u>	If the specified key is not already associated with a value or is associated with null, associate it with the given non-null value.

Methods	Action Performed
<u>putIfAbsent(K key, V value)</u>	Adds a mapping only if the key is not already mapped.

2. List Implementations

Lists represent ordered collections that allow duplicate elements and positional access. They include dynamic arrays, linked structures and legacy classes designed for sequential storage.

- [ArrayList](#)
- [LinkedList](#)
- [Vector](#)
- [Stack](#)
- [AbstractList](#)
- [AbstractSequentialList](#)

ArrayList in Java

Last Updated : 17 Mar, 2026

- ArrayList in Java is a resizable array provided in the [java.util package](#). Unlike normal [arrays](#), its size can grow or shrink dynamically as elements are added or removed.
 - Elements can be accessed using their indices, similar to arrays.
 - Duplicates are allowed.
 - Elements are stored in the order they are inserted.
 - ArrayList is not thread-safe. To make it thread-safe, you must wrap it manually using [Collections.synchronizedList\(\)](#).



```
import java.util.ArrayList;

class Main {
    public static void main (String[] args) {

        // Creating an ArrayList
        ArrayList<Integer> a = new ArrayList<Integer>();

        // Adding Element in ArrayList
        a.add(1);
        a.add(2);
        a.add(3);

        // Printing ArrayList
        System.out.println(a);
    }
}
```

Output

```
[1, 2, 3]
```

Explanation: This program creates an ArrayList of integers, adds elements to it using the add() method, and stores them dynamically. Finally, it prints the elements in insertion order as [1, 2, 3].

Note: ArrayList cannot hold primitive types like int, double, or char directly. we must use their wrapper classes instead:

- Use Integer instead of int
- Use Double instead of double
- Use Character instead of char

Hierarchy of ArrayList

It implements [List Interface](#) which is a sub-interface of [Collection Interface](#).



ArrayList Constructors in Java

Java provides multiple constructors to create an ArrayList based on different requirements:

1. ArrayList()

Creates an empty ArrayList with default initial capacity.

```
ArrayList<Integer> arr = new ArrayList<>();
```

2. ArrayList(Collection<? extends E> c)

Creates an ArrayList initialized with elements from the specified collection.

```
ArrayList<String> arr = new ArrayList<>(collection);
```

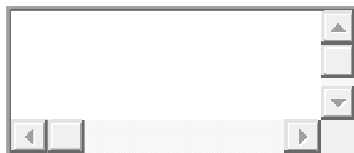
3. ArrayList(int initialCapacity)

This constructor is used to build an array list with the initial capacity being specified.

```
ArrayList<Double> arr = new ArrayList<>(20);
```

Operations of ArrayList

Now, Using the constructors we have got ArrayList for further operations like Insertion, Deletion and Updation of the elements in ArrayList.



```
import java.util.*;
```

```

class GFG{

    public static void main(String args[]){

        // Creating an Array of string type
        ArrayList<String> al = new ArrayList<>();

        // 1. Adding elements to ArrayList at the end
        al.add("Geeks");
        al.add("Geeks");

        System.out.println("Original List : "+al);

        // Adding Elements at the specific index
        al.add(1, "For");

        System.out.println("After Adding element at index 1 : "+
al);

        // 2. Removing Element using index
        al.remove(0);

        System.out.println("Element removed from index 0 : "+
al);

        // Removing Element using the value
        al.remove("Geeks");

        System.out.println("Element Geeks removed : "+ al);

        // 3. Updating value at index 0
        al.set(0, "GFG");

        System.out.println("List after updation of value :
"+al);
    }
}

```

[Try it on GfG Practice](#)

Output

```
Original List : [Geeks, Geeks]
After Adding element at index 1 : [Geeks, For, Geeks]
Element removed from index 0 : [For, Geeks]
Element Geeks removed : [For]
List after updation of value : [GFG]
```

3. Set Implementations

Sets represent collections of unique elements, disallowing duplicates. They provide implementations with different ordering strategies like hashing, insertion order or sorting.

- [HashSet](#)
- [Linked HashSet](#)
- [TreeSet](#)
- [EnumSet](#)
- [SortedSet Interface](#)
- [NavigableSet](#)
- [ConcurrentSkipListSet](#)

4. Queue / Deque Implementations

Queues store elements in a FIFO manner, while Deques allow operations at both ends. They are used for scheduling, buffering and producer-consumer applications.

- [Priority Queue](#)
- [Array Deque](#)
- [BlockingQueue](#)
- [ConcurrentLinked Queue](#)
- [Abstract Queue](#)

5. Map Implementations

Maps store data as key-value pairs where keys are unique. Different implementations provide hashing, ordering, reference-based and concurrent behaviors.

- [HashMap](#)
- [LinkedHashMap](#)
- [TreeMap](#)
- [WeakHashMap](#)
- [IdentityHashMap](#)
- [HashTable](#)